Sekolah Vokasi IPB
Kampus IPB Cilibende
Jl. Kumbang No.14, RT.02/RW.06, Babakan,
Kecamatan Bogor Tengah,
Kota Bogor, awa Barat 16128

Keamanan Perangkat Lunak (TPL2202)

Bahan Ajar

Penyusun:
Muhammad Nasir, S.T., M.Kom.
Lathifunnisa Fathonah, S.ST., M.T.
Donny Seftyanto, S.Tr.TP., M.T.
Dendi Furnama, S.T., M.Kom.

Editor:
Dr. Medhanita Dewi Renanti, S.Kom., M.Kom.

Teknologi Rekayasa Perangkat Lunak
Semester Ganjil
Tahun Akademik 2026/2027

Kata Pengantar

Puji dan syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya sehingga buku bahan ajar mata kuliah Keamanan Perangkat Lunak (TPL2202) ini dapat disusun dan diselesaikan dengan baik. Bahan ajar ini disusun sebagai upaya untuk mendukung proses pembelajaran yang terarah dan komprehensif dalam membekali mahasiswa dengan pengetahuan dan keterampilan terkait aspek keamanan dalam pengembangan perangkat lunak.

Mata kuliah Keamanan Perangkat Lunak memiliki peran penting dalam membentuk kompetensi mahasiswa agar mampu memahami berbagai ancaman, kerentanan, serta mekanisme pengamanan pada siklus hidup perangkat lunak. Oleh karena itu, materi dalam bahan ajar ini dirancang mencakup konsep teoritis dan praktik penerapan keamanan, mulai dari pengenalan prinsip keamanan, analisis risiko, pengamanan kode program, hingga pengujian dan evaluasi keamanan perangkat lunak.

Buku bahan ajar ini disusun oleh Muhammad Nasir, S.T., M.Kom. Lathifunnisa Fathonah, S.ST., M.T., Donny Seftyanto, S.Tr.TP., M.T., dan Donny Seftyanto, S.Tr.TP., M.T. serta disunting oleh Dr. Medhanita Dewi Renanti, S.Kom., M.Kom.. Penyusunan bahan ajar ini diharapkan dapat menjadi referensi utama bagi mahasiswa dalam proses pembelajaran, baik pada kegiatan perkuliahan maupun praktikum.

Kami menyadari bahwa bahan ajar ini masih memiliki keterbatasan. Oleh karena itu, kritik dan saran yang konstruktif sangat diharapkan sebagai bahan perbaikan dan pengembangan di masa yang akan datang. Semoga buku bahan ajar ini dapat memberikan manfaat dan kontribusi positif dalam meningkatkan kualitas pembelajaran serta kompetensi mahasiswa di bidang keamanan perangkat lunak.

Bogor,
Penyusun

Tim Penyusun

Penyusun



Muhammad Nasir, S.T., M.Kom.
NIDN : 0427119302
Instansi : Institut Pertanian Bogor
Unit Kerja : Sekolah Vokasi
Program Studi : Teknologi Rekayasa Perangkat Lunak
Email : m_nasir@apps.ipb.ac.id



Lathifunnisa Fathonah, S.ST., M.T.
NUPTK : 5638771672230332
Instansi : Institut Pertanian Bogor
Unit Kerja : Sekolah Vokasi
Program Studi : Teknologi Rekayasa Komputer
Email : lathifunnisa@apps.ipb.ac.id

Donny Seftyanto, S.Tr.TP., M.T.

Dendi Furnama, S.T., M.Kom.

Editor



Dr. Medhanita Dewi Renanti, S.Kom., M.Kom.
NIDN : 0005128303
Instansi : Institut Pertanian Bogor
Unit Kerja : Sekolah Vokasi
Program Studi : Teknologi Rekayasa Perangkat Lunak
Email : medhanita@apps.ipb.ac.id

Daftar Isi

Kata Pengantar	2
Tim Penyusun	3
Daftar Isi	4
Modul 1	6
Modul 2	17

1

Dasar Keamanan Perangkat Lunak

Sub-CPMK1: Kompeten menganalisis konsep dasar keamanan perangkat lunak, prinsip keamanan informasi, ancaman, kerentanan, dan risiko keamanan perangkat lunak secara logis, sistematis, dan bertanggung jawab. (C4, P3, A3)

Modul 1

Dasar Keamanan Perangkat Lunak

Tujuan:

1. Memahami konsep dasar CIA Triad (Confidentiality, Integrity, Availability). Mengenal prinsip dasar enkripsi sebagai mekanisme pengamanan data.
2. Melakukan simulasi enkripsi dan dekripsi teks menggunakan Caesar Cipher.
3. Menerapkan enkripsi pada file dokumen (Microsoft Word).
4. Melakukan simulasi enkripsi menggunakan PGP.
5. Membuat replika domain menggunakan teknik phising.

Instruksi:

1. Awali setiap aktivitas anda dengan doa.
2. Pahami tujuan, dasar teori, dan latihan-latihan praktikum dengan baik.
3. Kerjakan tugas-tugas praktikum dengan baik, jujur, dan sabar.
4. Tanyakan kepada dosen dan asisten apabila ada hal-hal yang kurang jelas.

Alat yang Digunakan:

1. PC/laptop/media yang memenuhi kapasitas untuk instalasi dan menjalankan Microsoft Project.
2. Laptop/PC dengan Microsoft Word
3. Browser untuk aplikasi online:
   a. Caesar Cipher Tool → https://www.dcode.fr/caesar-cipher
   b. AES Enkripsi / Dekripsi → https://www.devglan.com/online-tools/aes-encryption-decryption
   c. MD5 & SHA Generator → https://www.md5hashgenerator.com/
4. Modul/handout materi yang akan di praktikkan.

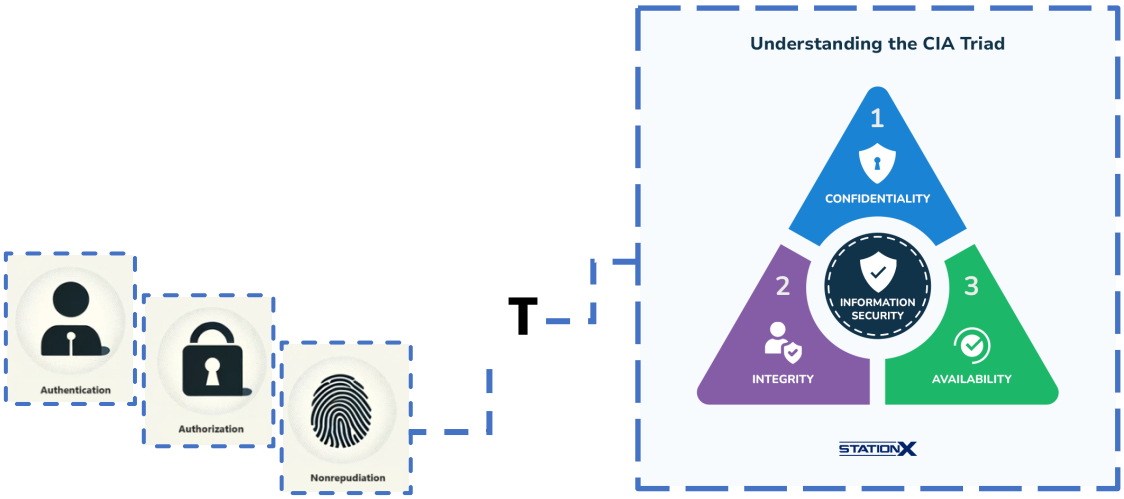
Dasar Teori:

1.1. Konsep Dasar Keamanan Perangkat Lunak

CIA Triad merupakan model fundamental dalam keamanan informasi yang terdiri dari tiga aspek utama:

- **Confidentiality (Kerahasiaan)**: Menjamin bahwa informasi hanya dapat diakses oleh pihak yang berwenang. Contoh: penggunaan password atau enkripsi.
- **Integrity (Integritas)**: Memastikan data tidak diubah, dimodifikasi, atau dirusak oleh pihak yang tidak berwenang. Contoh: penggunaan hashing atau digital signature.

- **Availability (Ketersediaan)**: Memastikan sistem dan data dapat diakses kapan pun oleh pengguna yang berhak. Contoh: backup data, sistem redundansi, dan server yang andal.
  Sumber: Stallings, W. (2018). Cryptography and Network Security: Principles and Practice (7th Edition). Pearson.



Konsep Dasar Keamanan

Keamanan sendiri merupakan upaya melindungi perangkat lunak dari akses, penggunaan, modifikasi, atau penghancuran yang tidak sah sehingga perangkat lunak dapat berjalan sesuai fungsi yang diharapkan.

1.2. Ancaman & Kerentanan

Ancaman adalah segala hal yang berpotensi merusak atau menyerang perangkat lunak.

Ancaman Pasif (Passive Threats)

- Penyadapan komunikasi (sniffing, eavesdropping).
- Pengumpulan informasi tanpa modifikasi data.

Ancaman Aktif (Active Threats)

- Injection Attack (SQL Injection, Command Injection).
- Cross-Site Scripting (XSS).
- Denial of Service (DoS/DDoS).
- Malware (virus, worm, trojan, ransomware).
- Man-in-the-Middle (MitM) → penyerang berada di antara

Insider Threats

- Serangan dari orang dalam (pegawai, admin) yang memiliki akses sah.

Ancaman Sosial (Social Engineering)

- Phishing, baiting, pretexting.

Kerentanan adalah kelemahan dalam perangkat lunak yang bisa dieksploitasi oleh ancaman.

- Kerentanan umum pada perangkat lunak meliputi:
- Kesalahan kode seperti buffer overflow
- Validasi input yang lemah
- Konfigurasi sistem yang salah
- Kurangnya enkripsi data
- Keterlambatan update atau patch keamanan

Keamanan penting dilakukan untuk:

- Melindungi data sensitif pengguna dan organisasi.
- Menjaga kepercayaan pengguna.
- Mencegah kerugian finansial akibat serangan.
- Memenuhi regulasi dan standar keamanan (misalnya GDPR, ISO/IEC 27001, OWASP).

1.3. Enkripsi Dasar

Enkripsi adalah proses mengubah plaintext menjadi ciphertext agar tidak dapat dibaca oleh pihak yang tidak memiliki kunci.

Enkripsi: proses mengubah plaintext menjadi ciphertext dengan algoritma tertentu.

Dekripsi: proses mengembalikan ciphertext ke plaintext.

Tujuan utama: menjaga kerahasiaan, integritas, dan keamanan komunikasi digital.

- Plaintext (Teks Asli / Polos): Pesan atau data asli yang masih dapat dibaca secara normal oleh manusia. Contoh: 'budi123' atau 'Data Rekam Medis'.
- Ciphertext (Teks Sandi): Pesan atau data yang sudah diacak menggunakan rumus matematika tertentu sehingga tidak bisa dibaca lagi secara langsung. Contoh: 'U2FsdGVkX19...'.
- Key (Kunci Rahasia): Nilai rahasia (bisa berupa angka, kata, atau string biner) yang digunakan oleh rumus matematika untuk mengunci (proses enkripsi) atau membuka (proses dekripsi) sebuah data.
- Encryption (Enkripsi): Proses mengubah data polos (Plaintext) menjadi data acak (Ciphertext) menggunakan Kunci Rahasia agar isinya tidak dapat diintip oleh pihak lain.
- Decryption (Dekripsi): Proses mengembalikan kembali data acak (Ciphertext) menjadi data polos semula (Plaintext) menggunakan Kunci Rahasia yang cocok.
- Hashing: Proses pembuatan 'sidik jari digital' dari suatu data. Fungsi Hash bersifat satu arah, artinya kita bisa mengubah data asli menjadi nilai Hash,

tetapi secara teknologi mustahil mengembalikan nilai Hash tersebut menjadi data aslinya.

- Salt (Garam Pengacak): String atau teks acak unik yang digabungkan ke password pengguna sebelum dilakukan proses Hashing. Tujuannya adalah untuk memastikan bahwa dua pengguna dengan password yang sama tetap memiliki nilai Hash akhir yang berbeda di sistem penyimpanan.
- RAM (Random Access Memory) / Dictionary Python: Wadah penyimpanan data sementara di komputer. Pada praktikum ini, kita menggunakan struktur data Kamus (Dictionary) Python di RAM sebagai pengganti database agar lebih mudah dipelajari oleh pemula.

a. Algoritma Klasik

Caesar Cipher → menggantikan huruf dengan pergeseran tertentu.

Vigenère Cipher → menggunakan kata kunci untuk menentukan pergeseran huruf.

Caesar Cipher adalah cara mengunci pesan yang sangat tua dan sederhana Caranya adalah dengan menggeser huruf asli sebanyak beberapa langkah ke depan dalam urutan abjad (A sampai Z).

Rumus Enkripsi (Geser Kanan): Huruf Baru = (Huruf Asli + Geser) dibagi sisa abjad (26).

Kelemahan: Sangat mudah ditebak karena jumlah abjad hanya ada 26 huruf. Penyerang tinggal mencoba menggeser dari 1 sampai 25 (disebut teknik Brute Force) untuk menemukan pesan aslinya.

b. Algoritma Modern

AES (Advanced Encryption Standard) → block cipher 128-bit, kunci 128/192/256 bit.

Karena enkripsi klasik sangat mudah ditembus, sistem komputer modern menggunakan AES. AES menggunakan matematika yang jauh lebih rumit sehingga komputer tercepat di dunia pun butuh waktu miliaran tahun untuk menebak kuncinya tanpa izin. Di Python, kita akan menggunakan cara termudah dan teraman bernama Fernet (bagian dari pustaka cryptography) yang menggunakan standar AES.

DES (Data Encryption Standard) → block cipher 64-bit, dianggap lemah saat ini.

RSA (Rivest-Shamir-Adleman) → algoritma asimetris berbasis faktorisasi bilangan prima besar.

ECC (Elliptic Curve Cryptography) → keamanan tinggi dengan panjang kunci lebih pendek.

c. Fungsi Hash

MD5 → menghasilkan hash 128-bit (rentan terhadap collision).

SHA (Secure Hash Algorithm) → SHA-256, SHA-512 digunakan untuk keamanan modern.

Mengapa Password Tidak Boleh Disimpan Langsung?
Jika kita membuat aplikasi, kita tidak boleh menyimpan password user dalam bentuk teks asli. Mengapa? Karena jika ada orang lain yang melihat data tersebut, password akan langsung bocor.
Untuk mengamankannya, kita gunakan fungsi Hash (SHA-256). Fungsi hash bertindak seperti mesin penghancur satu arah: ia bisa mengubah password menjadi "sidik jari acak" sepanjang 64 karakter, tapi sidik jari itu tidak bisa dikembalikan lagi menjadi password asli.

Masalahnya: Jika dua user menggunakan password yang sama (misalnya: rahasia123), maka hasil hash mereka akan sama persis. Hacker bisa menebak password ini menggunakan daftar kata-kata pasaran yang sudah di-hash sebelumnya (disebut Rainbow Table).

Solusinya (Salt): Sebelum password di-hash, kita tambahkan teks acak yang unik untuk setiap user (teks acak ini disebut Salt). Dengan begitu, meskipun passwordnya sama, hasil hash akhirnya akan berbeda jauh karena dicampur dengan Salt yang berbeda.

Praktikum:

1. **Enkripsi Manual**
   Kerjakan latihan di bawah ini di lembar kertas atau buku catatan Anda.
   Latihan Caesar Cipher Manual
   Mari kita enkripsi **kata** "IPB" dengan **kunci** pergeseran 3.
   Langkah 1: Ubah huruf menjadi angka (A=0, B=1, C=2, ..., I=8, ..., P=15, ...).
   I = indeks ke-8
   P = indeks ke-15
   B = indeks ke-1
   Langkah 2: Tambahkan kunci pergeseran (3) ke masing-masing angka [10].
   Huruf Baru I = $(8 + 3) = 11$
   Huruf Baru P = $(15 + 3) = 18$
   Huruf Baru B = $(1 + 3) = 4$
   Langkah 3: Kembalikan angka baru menjadi huruf abjad [10].
   Indeks 11 = L
   Indeks 18 = S
   Indeks 4 = E
   Hasil Akhir **Plaintext** "IPB" menjadi **Ciphertext**: "LSE"

Tugas Anda di Kertas: Dekripsikan kata "LSE" kembali menjadi pesan asli dengan cara mengurangi indeks huruf dengan kunci pergeseran (3)! Tulis jalannya di laporan praktikum.

Latihan Konsep Salted Hashing Manual

Bayangkan Anda memiliki dua pengguna dengan password yang sama:

Andi, Password: budi123
Siti, Password: budi123
Jika kita hanya memakai SHA-256 biasa tanpa Salt, hash mereka di penyimpanan akan sama persis. Sekarang mari kita tambahkan Salt acak:

Andi diberi Salt: xyz789
Siti diberi Salt: abc111
Maka data yang akan di-hash oleh sistem adalah:

Data Andi = xyz789 + budi123 = xyz789budi123
Data Siti = abc111 + budi123 = abc111budi123
Pertanyaan Analisis: Apakah hasil akhir hash Andi dan Siti akan sama setelah digabungkan dengan Salt? Jelaskan alasan Anda berdasarkan konsep keunikan Salt!

2. **Enkripsi Dasar pada File Word**
   a. Buka Microsoft Word.
   b. Buat dokumen baru dan tuliskan teks: *"Keamanan Informasi adalah hal penting"*.
   c. Simpan dokumen dengan nama praktikum.docx.
   d. Klik **File** → **Info** → **Protect Document** → **Encrypt with Password**.
   e. Masukkan password misalnya 12345.
   f. Tutup lalu buka kembali dokumen, pastikan password diminta.

3. **Enkripsi Online – dCode Caesar Cipher**
   a. Buka situs:
   https://www.dcode.fr/caesar-cipher
   b. Masukkan teks KEAMANAN.
   c. Pilih shift = 5.
   d. Klik **Encrypt** → catat hasil ciphertext.
   e. Coba gunakan fitur **Decrypt (Bruteforce)** untuk mengembalikan plaintext tanpa tahu kunci.

Caesar Cipher dalam bentuk manual:

Posisi	Huruf Asli	→	Huruf Setelah Geser (+3) (Ciphertext)
1	K	→	N
2	E	→	H
3	A	→	D
4	M	→	P
5	A	→	D
6	N	→	Q
7	A	→	D
8	N	→	Q

4. Melakukan Simulasi Enkripsi plaintext menggunakan PGP

a. Download Cleopatra di situs https://gpg4win.org/download.html
b. Install di komputer anda
c. Buka cleopatra lalu generate key pair
d. Buat nama dan email yang sesuai
e. Simpan private key anda bisa di notepad
f. Lalu buat pesan berupa plaintext yang akan dikirmkan di notepad dengan di ketik
g. Copy file tersebut
h. Buka tools di cleopatra, lalu clipboard dan pilih encrypt
i. Setelah itu paste Kembali di notepad
j. Pesan telah berhasil di enkripsi
k. Untuk mendekripsinya lakukan hal yang sama copy file yang di enkripsi
l. Lalu pilih tools, clipboard dan decrypt
m. Setelah itu cek dengan paste di note pad Kembali maka akan menghasilkan pesan yang sebenarnya

5. Caesar Cipher - Membuat Program Pengacak dan Brute Force

Mari kita buat program sederhana untuk mengotomatiskan pergeseran huruf Caesar Cipher, sekaligus mencoba teknik Brute Force (mencoba semua kemungkinan kunci dari 1 sampai 25) untuk memecahkan sandi tersebut tanpa tahu kuncinya.

1. Buat berkas baru di VS Code dengan nama percoba_1_caesar.py
2. Tuliskan kode Python sederhana di bawah ini:

```
def enkripsi_caesar(teks_asli, geser):
    """Menggeser huruf plaintext sesuai nilai kunci."""
    hasil = ""
    for i in range(len(teks_asli)):
        char = teks_asli[i]

        # Enkripsi huruf kapital
        if char.isupper():
            hasil += chr((ord(char) + geser - 65) % 26 + 65)
        # Enkripsi huruf kecil
        elif char.islower():
            hasil += chr((ord(char) + geser - 97) % 26 + 97)
        # Jika karakter berupa angka, spasi, atau simbol, biarkan
utuh
        else:
            hasil += char
    return hasil

def dekripsi_caesar(teks_sandi, geser):
    """Mendekripsi kembali dengan geseran berlawanan arah."""
    return enkripsi_caesar(teks_sandi, -geser)

def brute_force_caesar(teks_sandi):
    """Mencoba menebak seluruh kemungkinan kunci (1 s.d. 25)."""
    print("\n=== MEMULAI SERANGAN BRUTE FORCE ===")
    for kunci_coba in range(1, 26):
        hasil_coba = dekripsi_caesar(teks_sandi, kunci_coba)
        print(f"Dicoba Kunci {kunci_coba:02d} -> Hasil Dekripsi:
{hasil_coba}")

# --- SKENARIO UJI COBA JALANNYA PROGRAM ---
if __name__ == "__main__":
    pesan_asli = "Praktikum TRPL IPB University"
    kunci_rahasia = 5

    # 1. Proses Mengubah ke Ciphertext
    teks_terkunci = enkripsi_caesar(pesan_asli, kunci_rahasia)
    print(f"Pesan Asli : {pesan_asli}")
    print(f"Kunci Geser: {kunci_rahasia}")
    print(f"Hasil Sandi: {teks_terkunci}")

    # 2. Proses Mengembalikan ke Plaintext
    teks_terbuka = dekripsi_caesar(teks_terkunci, kunci_rahasia)
    print(f"Dekripsi   : {teks_terbuka}")

    # 3. Simulasi serangan tebak kunci oleh hacker
    brute_force_caesar(teks_terkunci)
```

Dari hasil serangan Brute Force di layar terminal Anda, pada baris ke berapa pesan rahasia berhasil terbongkar oleh hacker? Apakah metode Caesar Cipher ini masih layak digunakan untuk melindungi data website di internet saat ini? Jelaskan alasannya!

6. Enkripsi Modern AES untuk Data Rekam Medis Pasien

Sekarang kita beralih ke enkripsi tingkat industri modern yaitu AES-128 menggunakan skema Fernet yang aman dan diakui secara global.

Buka terminal/command prompt di komputer Anda, kemudian install library keamanan standar Python:

```
pip install cryptography
```

Buat berkas baru bernama percoba_2_aes.py

Tuliskan kode program di bawah ini:

```
from cryptography.fernet import Fernet

def buat_kunci_baru(nama_file_kunci):
    """Membuat kunci AES baru dan menyimpannya di file eksternal."""
    kunci = Fernet.generate_key()
    with open(nama_file_kunci, "wb") as file_kunci:
        file_kunci.write(kunci)
    print(f"Kunci pengunci aman telah dibuat di file:
{nama_file_kunci}")

def ambil_kunci_dari_file(nama_file_kunci):
    """Membaca kunci AES yang tersimpan."""
    return open(nama_file_kunci, "rb").read()

def enkripsi_aes(teks_asli, kunci):
    """Mengenkripsi teks asli menjadi teks sandi biner AES."""
    mesin_kripto = Fernet(kunci)
    # Teks harus diubah menjadi bentuk Byte terlebih dahulu (encode)
    teks_byte = teks_asli.encode('utf-8')
    teks_sandi_aes = mesin_kripto.encrypt(teks_byte)
    return teks_sandi_aes

def dekripsi_aes(teks_sandi_aes, kunci):
    """Mendekripsi teks sandi AES kembali menjadi string teks
asli."""
    mesin_kripto = Fernet(kunci)
    teks_byte_terbuka = mesin_kripto.decrypt(teks_sandi_aes)
    return teks_byte_terbuka.decode('utf-8')

# --- SKENARIO UJI COBA JALANNYA PROGRAM ---
if __name__ == "__main__":
    file_kunci = "kunci_rahasia.key"
    data_pasien = "Rekam Medis: Pasien Atas Nama budi, menderita
Demam Berdarah (DBD)."

    # 1. Hasilkan Kunci Enkripsi AES Baru
    buat_kunci_baru(file_kunci)
    kunci_aes = ambil_kunci_dari_file(file_kunci)
    print(f"Kunci AES Anda (Base64): {kunci_aes.decode('utf-8')}")

    # 2. Proses Enkripsi
    sandi_terkunci = enkripsi_aes(data_pasien, kunci_aes)
    print(f"\nPlaintext  : {data_pasien}")
```

```
    print(f"Ciphertext : {sandi_terkunci.decode('utf-8')}")

    # 3. Proses Dekripsi
    data_kembali = dekripsi_aes(sandi_terkunci, kunci_aes)
    print(f"Hasil Balik: {data_kembali}")
```

Mengapa kunci AES di atas disimpan di berkas eksternal (kunci_rahasia.key) dan bukan ditulis langsung di baris kode program utama Anda? Jelaskan potensi bahayanya bagi keamanan aplikasi jika kunci ditulis keras (hardcoded) di dalam program!

Tugas

Carilah 1 paragraf minimal 60 kata dari suatu berita yang berhubungan dengan serangan siber, lakukan enkripsi untuk 1 paragraf tersebut menggunakan Teknik Caesar Cipher (tentukan kunci pergeseran dengan aturan kunci = 1 angka terakhir pada NIM masing-masing, untuk angka akhir 0 gunakan angka akhir pada NIM di atas NIM anda), tulis tangan jawaban di kertas A4 dan dikumpulkan pada drive pengumpulan tugas praktikum.

Contoh kunci pergeseran:

Daftar NIM

ABC02192

ABC02199

ABC02200

NIM : ABC02192, kunci = 2

NIM : ABC02200, kunci = 9

2

Manajemen Risiko dan Analisis Ancaman

Sub-CPMK1: Kompeten menganalisis konsep dasar keamanan perangkat lunak, prinsip keamanan informasi, ancaman, kerentanan, dan risiko keamanan perangkat lunak secara logis, sistematis, dan bertanggung jawab. (C4, P3, A3)

Modul 2

Manajemen Risiko & Analisis Ancaman

(Risk Identification & Threat Modeling)

Tujuan:

1. Memahami konsep dasar manajemen risiko dalam keamanan perangkat lunak.
2. Mengetahui langkah-langkah identifikasi risiko dan analisis ancaman.
3. Mengenal metode threat modeling (STRIDE & Attack Tree).
4. Membuat tabel analisis risiko sederhana dari studi kasus.
5. Memvisualisasikan ancaman dalam bentuk attack tree.

Instruksi:

1. Awali setiap aktivitas anda dengan doa.
2. Pahami tujuan, dasar teori, dan latihan-latihan praktikum dengan baik.
3. Kerjakan tugas-tugas praktikum dengan baik, jujur, dan sabar.
4. Tanyakan kepada dosen dan asisten apabila ada hal-hal yang kurang jelas.

Alat yang Digunakan:

1. PC/laptop/media yang memenuhi kapasitas untuk instalasi dan menjalankan Microsoft Project.
2. Laptop/PC dengan Microsoft Word
3. Browser untuk identifikasi sistem/aplikasi berbasis web
4. Modul/handout materi yang akan di praktikkan.

Dasar Teori:

1. **Manajemen Risiko**

   Manajemen risiko keamanan informasi adalah proses sistematis untuk mengidentifikasi, menilai, dan mengendalikan risiko. Tahapan utama:

   1) Identifikasi risiko → menentukan ancaman & kerentanan.
   2) Analisis risiko → menilai dampak (impact) dan kemungkinan (likelihood).
   3) Evaluasi risiko → menentukan prioritas risiko.
   4) Mitigasi risiko → merancang kontrol pengamanan.

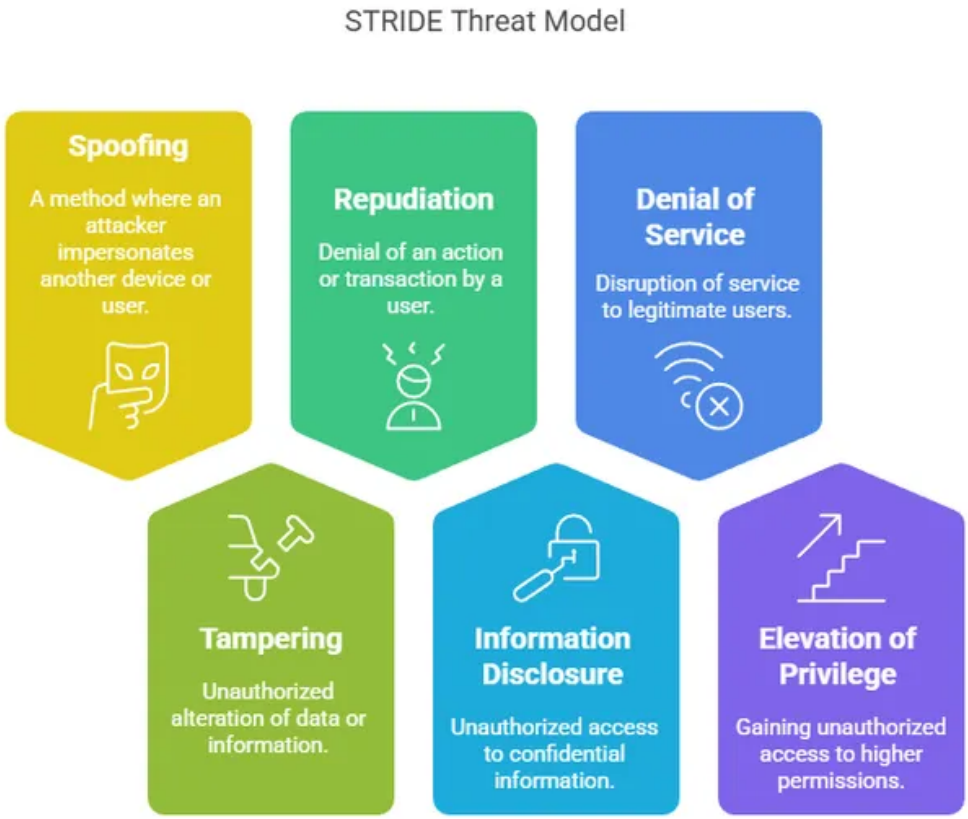
2. **Threat Modeling**

   Threat modeling adalah pendekatan terstruktur untuk menganalisis potensi ancaman terhadap sistem.

   STRIDE: **S**poofing, **T**ampering, **R**epudiation, **I**nformation Disclosure, **D**enial of Service, **E**levation of Privilege.

PASTA: **P**rocess for **A**ttack **S**imulation and **T**hreat **A**nalysis.
Attack Trees: Diagram yang memetakan cara penyerang mencapai tujuan.

Threat	Desired Security Property
Spoofing	Authentication
Tampering	Integrity
Repudiation	Non-repudiation
Information Disclosure	Confidentiality
Denial of Service	Availability
Elevation of Privilege	Authorization



Praktikum

1. **Identifikasi Risiko**

   Studi Kasus: Sistem E-commerce.

Ancaman	Kerentanan	Dampak	Likelihood	Risk Level	Mitigasi
SQL Injection	Input form tidak divalidasi	Data pelanggan bocor	Tinggi	Sangat Tinggi	Validasi input, parameterized SQL
DDoS Attack	Tidak ada proteksi server	Layanan tidak tersedia	Sedang	Tinggi	Firewall, CDN, rate limiting
Insider Threat	Password default tidak diganti	Penyalahggunaan akses	Rendah	Sedang	Kebijakan keamanan internal

2. **STRIDE (Analisis Ancaman)**

   Langkah 1: Identifikasi Komponen Sistem

   User -> Browser -> Web Server -> Database

   Langkah 2: Analisis Ancaman per Kategori STRIDE

   Tabel Contoh Ancaman per Kategori STRIDE

Kategori	Deskripsi	Contoh Ancaman	Mitigasi
Spoofing	Penyamaran identitas pengguna	Penyerang login menggunakan akun orang lain	Multi-factor authentication, OTP
Tampering	Modifikasi data tanpa izin	Perubahan query SQL (SQL Injection)	Input validation, parameterized query
Repudiation	Penyangkalan tindakan yang sudah dilakukan	User menyangkal sudah melakukan transaksi	Logging, digital signature
Information Disclosure	Kebocoran data rahasia	Password disimpan dalam plaintext	Enkripsi, hashing (bcrypt, Argon2)
Denial of Service	Mengganggu atau menghentikan layanan	Serangan DDoS pada server	Firewall, rate limiting, CAPTCHA
Elevation of Privilege	Mendapat hak akses lebih tinggi dari seharusnya	User biasa menjadi admin	Role-based access control, least privilege

Tabel Analisis Ancaman STRIDE per Komponen

Komponen	STRIDE	Contoh Ancaman	Dampak	Mitigasi Praktis
User	**S**poofing	Penyerang menggunakan akun orang lain	Akses ilegal	Gunakan OTP / 2FA
Form Login	**T**ampering	Mengubah request login (SQL Injection)	Data bocor	Validasi input, parameterized query
Server Log	**R**epudiation	User menyangkal sudah melakukan login	Hilang jejak audit	Simpan log dengan digital signature
Database	**I**nformation Disclosure	Password disimpan plaintext	Kredensial bocor	Simpan dengan hashing (bcrypt/argon2)
Web Server	**D**enial of Service	Brute force / DDoS	Layanan down	Rate limiting, firewall, CAPTCHA
Database	**E**levation of Privilege	User biasa naik jadi admin	Kontrol penuh sistem	RBAC, least privilege access

3. Attack Tree pada Login

Representasi visual untuk menggambarkan cara-cara yang dapat digunakan penyerang mencapai tujuan. Tujuan dari Attack tree ini memungkinkan penyerang mendapatkan akses ilegal ke akun pengguna.



Brute Force Attack → mencoba banyak password.
Phishing → menipu user agar memberi password.
SQL Injection → memanipulasi query login.

Kelebihan Attack Tree:

a. Mempermudah identifikasi jalur serangan.

b. Bisa digunakan untuk menilai risiko (probabilitas + dampak).
c. Mendukung prioritisasi mitigasi.

Tugas

1. Tugas Berkelompok
2. Definisikan dengan bahasa sendiri apa itu manajemen risiko dalam keamanan informasi.
3. Buat matriks risiko dari minimal 3 ancaman pada sistem informasi akademik kampus (IPBM, Student Portal, dll).
4. Lakukan analisis STRIDE untuk sistem pada poin ke 3.
5. Gambarkan attack tree dengan tujuan: Penyerang mendapatkan akses database mahasiswa.